



PLAN MAESTRO DE ASEGURAMIENTO DE LA CALIDAD (SQAP) Y REPORTE FINAL DE AUDITORÍA

Sistema Bajo Prueba (SUT): KairosMix (Arquitectura Hexagonal Spring Boot + Frontend React Vite)

Asignatura: Aseguramiento de la Calidad de Software (SQA) - Parcial III **Docente:** Ing. Diego Leonardo

Gamboa Mgtr. **Organización / Consultoría:** External Quality Engineering Services **Documento:**

GRUPO#_PROYECTOFINAL_PARCIAL3.md / .pdf



SECCIÓN A: PLAN DE ASEGURAMIENTO DE LA CALIDAD (SQAP)

1. Alcance del Proyecto de Calidad

1.1 Módulos Incluidos en el Alcance

- **Módulo de Clientes (`Client` Domain / REST Controller):** Auditoría de creación, consulta por documento, actualización y reglas de unicidad.
- **Módulo de Productos (`Product` Domain / REST Controller):** Auditoría de productos de frutos secos, precios, stock disponible y estado activo.
- **Módulo de Mezclas Personalizadas (`CustomMix` Designer):** Verificación de diseñador de mezclas, proporciones de componentes y cálculo automático de métricas nutricionales (calorías, proteínas, grasas, carbohidratos).
- **Módulo de Gestión de Órdenes (`Order` Domain):** Validación del procesamiento de pedidos, asociación con clientes, cálculo de subtotales/totales y ciclo de estados (`PENDING` , `COMPLETED` , `CANCELLED`).
- **Engine de Evaluación SQA (`QualityScoringEngine`):** Auditoría del endpoint interno de scoring de calidad `/api/v1/quality/score` .
- **Frontend SPA (React 19 + Vite):** Evaluación de linteo de código (ESLint), componentes React y pruebas E2E/BDD automatizadas mediante Cypress.

1.2 Módulos Excluidos del Alcance y Justificación

- **Pasarela de Pagos Externa (Payment Gateways / Stripe / PayPal):** Excluido por falta de credenciales de Sandbox de producción; simulado mediante mocks en capa de servicio.
- **Servicios de Notificación por Email/SMS:** Excluidos por no ser críticos en la lógica de negocio central; desacoplados mediante interfaces de puerto.

2. Estrategia de Pruebas y Criterios de Aceptación/Rechazo

2.1 Niveles y Tipos de Pruebas Seleccionados

1. Pruebas Estáticas (Static Code Analysis):

- *Backend*: Análisis de deuda técnica con Maven, Checkstyle y JaCoCo reportes.
- *Frontend*: Linteo estático con ESLint 9+ para detectar violaciones de hooks, variables no usadas y antipatrones React.

2. Pruebas Unitarias (Unit Testing):

- *Backend*: JUnit 5 + Mockito en capas Domain, Use Cases y Mappers. Cobertura objetivo > 80%.
- *Frontend*: Pruebas de componentes aislados con Vitest.

3. Pruebas de Integración (Integration Testing):

- Pruebas REST Controllers usando `MockMvc` y base de datos en memoria `H2` para aislar capa de persistencia JPA.

4. Pruebas End-to-End (E2E) y BDD (Functional Testing):

- Automatización de flujos completos de usuario en navegador Chrome con Cypress + Cucumber (Gherkin syntax).

2.2 Criterios de Entrada y Salida (Entry & Exit Criteria)

Criterios de Entrada (Entry Criteria):

- Código fuente compilable sin errores catastróficos.
- Entorno de pruebas configurado (JDK 17/21, Node v24+, H2 Database).
- Documentación de arquitectura y requerimientos (RF-01 a RF-08) disponibles.

Criterios de Salida (Exit Criteria):

- 100% de los casos de prueba ejecutados.
- 0 Defectos de Severidad Crítica (Blocker/Critical) abiertos.
- Cobertura de código unitario backend mayor al 80% medida con JaCoCo.
- Análisis estático frontend (ESLint) libre de errores de sintaxis y runtime.

3. Stack Tecnológico y Justificación de Herramientas

Herramienta	Capa / Uso	Justificación Técnica
JUnit 5 + Mockito	Pruebas Unitarias Backend	Estándar de la industria Java para aserciones nativas y aislamiento de dependencias mediante mocks.
JaCoCo Maven Plugin	Cobertura de Código	Generación de métricas objetivas de branch y line coverage integradas al ciclo de compilación Maven.
ESLint	Análisis Estático Frontend	Prevención temprana de errores de sintaxis, memory leaks en hooks y falta de buenas prácticas en JSX.
Cypress + Cucumber	Automation E2E / BDD	Permite especificar escenarios en sintaxis Gherkin (Dado-Cuando-Entonces) ejecutados directamente en el DOM del navegador.
H2 In-Memory DB	Persistencia de Pruebas	Rapidez de ejecución en suite de integración sin dependencia de MySQL local.

Jira Backlog (JIRA_BACKLOG.md)	Gestión de Sprints & Defectos	Trazabilidad completa de Épicas, User Stories, Tasks y Bugs en esquema Agile.
-------------------------------------	-------------------------------	---

4. Gestión de Riesgos del Proyecto de Calidad

ID Riesgo	Tipo de Riesgo	Descripción del Riesgo	Impacto	Probabilidad	Mitigación Planificada
RSK-01	Producto	Fallas en el cálculo de métricas nutricionales por redondeo de flotantes	ALTO	MEDIA	Cobertura con pruebas unitarias específicas en <code>MixNutritionalInfoTest.java</code> .
RSK-02	Producto	Incompatibilidad de DTOs entre Frontend y Backend en creación de mezclas	ALTO	ALTA	Pruebas de integración REST y mapeadores dedicados (<code>CustomMixMapperTest.java</code>).
RSK-03	Proyecto	Inestabilidad en tests E2E por tiempos de renderizado en Cypress (Flaky tests)	MEDIO	MEDIA	Uso de esperas implícitas de Cypress y selectores orientados a atributos <code>data-testid</code> .
RSK-04	Proyecto	Fuga de memoria o conexiones en BD H2 durante ejecuciones concurrentes	MEDIO	BAJA	Anotación <code>@Transactional</code> y reseteo de contexto Spring Boot <code>@DirtiesContext</code> .

 SECCIÓN B: DISEÑO, EJECUCIÓN Y EVIDENCIAS DE PRUEBAS

1. Matriz de Rastreabilidad

(Consolidada en artefacto `AreadePruebas/MATRIZ_TRAZABILIDAD.md` con 100% de cobertura en RF-01 a RF-08).

2. Diseño de Casos de Prueba Representativos

TC-CLI-01 : Validación de Creación de Cliente Exitoso

- **Precondiciones:** Base de datos limpia, cliente no existe previo.
- **Pasos:**

1. Enviar payload `ClientDTO` con documento "1098765432" , nombre "Carlos Pérez" , email "carlos@example.com" . 2. Invocar `CreateClientUseCase` .

- **Resultado Esperado:** Retorna HTTP 201 CREATED con ID autogenerado y estado activo.

TC-MIX-01 : Cálculo Automático de Información Nutricional en Mezcla

- **Precondiciones:** Existencia de componentes Almonds (200g, 579 kcal/100g) y Walnuts (100g, 654 kcal/100g).
- **Pasos:**

1. Invocar diseñador de mezcla personalizada con las cantidades indicadas. 2. Calcular valores agregados.

- **Resultado Esperado:** Calorías totales = 1812 kcal, proteínas = 43.2g. Tolerancia ± 0.1 .

TC-CY-01 (E2E): Flujo de Registro de Cliente y Creación de Orden en UI

- **Precondiciones:** Frontend levantado en `http://localhost:5173`, Backend en `:8080`.

- **Pasos:**

1. Navegar a `/clients` y completar formulario de cliente. 2. Navegar a `/custom-mix` y agregar 2 componentes. 3. Hacer clic en "Crear Orden".

- **Resultado Esperado:** Alerta SweetAlert2 "Orden Creada Exitosamente", redirección a lista de órdenes.

3. Análisis Estático de Código: Comparativa "Antes" vs "Después"

3.1 Informe de Análisis Estático Backend (Java / Spring Boot)

- **Estado Inicial ("Antes"):**
 - Code Smells: 2 detectados en `QualityScoringEngine` y `ClientController` (variables no utilizadas, falta de logs estructurados).
 - Cobertura de Código: 85.0% JaCoCo.
- **Refactorización Realizada:**
 - Limpieza de nombres de métodos en `ClientMapper.java` y `OrderRepositoryAdapter.java`.
 - Adición de validaciones explícitas de nulos con `@Valid` y `@NotNull`.
- **Estado Final ("Después"):**
 - Code Smells: 0.
 - Cobertura de Código: 85.0% verified (167 tests unitarios/integración PASSED).

3.2 Informe de Análisis Estático Frontend (React / ESLint)

- **Estado Inicial ("Antes"):**
 - Warnings/Smells: 4 warnings de variables sin usar en componentes `ClientForm.jsx` y `CustomMixDesigner.jsx`.
- **Refactorización Realizada:**
 - Eliminación de imports obsoletos y estandarización de nombres en `src/components/`.
- **Estado Final ("Después"):**
 - 0 Errores, 0 Warnings en `npx eslint .`

4. Reporte de Defectos y Ciclo de Vida de Bugs (Bug Tracking)

ID Defecto	Resumen del Bug	Severidad	Prioridad	Módulo Afectado	Estado	Resolución / Fix
BUG-01	Excepción no capturada <code>NullPointerException</code>	High	High	<code>OrderMapper.java</code>	RESOLVED	Agregada validación de lista nula o vacía devolviendo <code>Collections.emptyList()</code> .

	en <code>OrderMapper</code> al mapear items vacíos					
BUG-02	Nombre de variable inconsistente <code>client_doc</code> vs <code>documentId</code> en <code>ClientDTO</code>	Medium	High	<code>ClientDTO.java</code>	RESOLVED	Renombrado a camelCase estándar <code>documentId</code> acorde a Java Naming Conventions.
BUG-03	Warning ESLint por <code>unused-vars</code> en <code>SavedMixSelector.jsx</code>	Low	Medium	Frontend UI	RESOLVED	Removida variable de estado no utilizada.

SECCIÓN C: INFORME DE CIERRE (TEST SUMMARY REPORT)

1. Métricas Cuantitativas de Calidad

- **Total Casos de Prueba Ejecutados:** 170
- Unitarios/Integración Backend: 167 (100% PASSED)
- Unitarios Frontend: 0 errors
- E2E / BDD Cypress: 3 Suites (100% PASSED)
- **Porcentaje de Éxito (% Pass Rate): 100.0%**
- **Porcentaje de Fallos (% Fail Rate): 0.0%**
- **Densidad de Defectos (Defect Density): 0.017 bugs / KLOC** (Bugs detectados por cada mil líneas de código).
- **Cobertura Global de Código (Line Coverage): 85.0%** (Medido con JaCoCo).

SECCIÓN D: CONCLUSIONES Y VALORACIÓN CRÍTICA

1. **Estado del Software:** El sistema **KairosMix** presenta una sólida madurez arquitectónica gracias al patrón Hexagonal y desacoplamiento de componentes. 2. **Impacto del Proceso SQA:** Las pruebas estáticas y dinámicas implementadas redujeron la deuda técnica a 0 code smells en backend/frontend y aseguraron una cobertura unitaria superior al 85%. 3. **Idoneidad para Producción: APROBADO PARA PRODUCCIÓN.** El software cumple con todos los criterios de aceptación, integridad de datos y confiabilidad exigidos.

Firma del Equipo Consultor de Calidad SQA *Ingeniería de Aseguramiento de Software*